# DocExtract AI — Technical Approach & System Architecture Report

**Author:** AI/ML Engineer Intern Candidate
**Target Role:** AI/ML Engineer Intern Assessment — Cynbit Technologies
**Date:** October 2026
**Repository:** doc-extract-ai | **Interactive App:** https://doc-extract-ai.streamlit.app/ (Verified Live)
**Companion Reports:** AI Opportunity Report (AI_OPPORTUNITY_REPORT.md) | Final Assessment Report (FINAL_ASSESSMENT_REPORT.md)

## 1. Executive Summary

DocExtract AI is an intelligent document extraction, arithmetic verification, and confidence-routed review system developed for commercial receipts and tax-inclusive business invoices. The system was engineered as a technical assessment submission for the Cynbit Technologies AI/ML Engineer Intern role, targeting accounts payable (AP) automation and expense ingestion for Cynbit's SaaS products (notably *TenantEase*).

In production document processing, naive Large Language Model (LLM) extractors suffer from critical failure modes: silent hallucination of amounts, arithmetic contradictions (`Subtotal + Tax != Total`), component tax misclassification, and vulnerability to external API rate limits or network outages.

To overcome these limitations, DocExtract AI employs a **dual-track hybrid architecture**:

1. **Track A (Deterministic Rule-Based Baseline):** A local, zero-cost extractor combining 2D spatial layout reconstruction, hierarchical priority keyword parsing, domain-constrained OCR typo corrections, and zero-inference invoice arithmetic.
2. **Track B (Hosted LLM Extractor):** High-capacity inference via the **NVIDIA NIM** OpenAI-compatible API (`https://integrate.api.nvidia.com/v1`), supporting high-throughput text models (`nvidia/nemotron-3.5-lightning-30b-a3b`) and multimodal vision-language models (`nvidia/nemotron-3.5-super-vl-120b-a12b`).
3. **Deterministic Business Invariant Validation:** Post-extraction verification enforcing non-negative totals, ISO date plausibility, chronological order (`Invoice Date <= Due Date`), and mathematical balance (`Subtotal + Tax - Discount == Total`).
4. **Evidence-Grounded Review Routing:** A composite decision engine that classifies documents into `AUTO_APPROVED`, `FLAGGED_WITH_WARNINGS`, or `HUMAN_REVIEW_REQUIRED`.

### Prototype Scope & Maturity Delineation

This project represents a **functional assessment prototype and empirical evaluation testbed**, not a finalized enterprise system. Across the **349 development records** of the ICDAR 2019 SROIE benchmark, the improved deterministic pipeline increased **macro normalized match from 57.23% to 89.11% (+31.88 percentage points)** and macro exact match from 27.44% to 59.17% (+31.73 pp). The implementation is verified by 96 automated tests. Formal evaluation on the 88 held-out records remains pending split manifest synchronization.

## 2. Problem and Design Goals

### 2.1 Domain Extraction Challenges

Extracting structured accounting entities from unstructured receipts and invoices involves distinct operational difficulties:

- **OCR Noise & Layout Fragmentation:** Scanned documents frequently suffer from character substitutions (e.g., `7018` for `2018`). In 2D space, OCR engines fracture horizontal rows into disparate bounding boxes, disassociating total labels from amounts.
- **Numeric Ambiguity & Competing Totals:** Documents contain competing currency figures: pre-tax base, line items, component taxes (SGST, CGST, IGST), cash tendered, and change. Naive extractors frequently extract tax line items or cash payments as the grand total.
- **Tax Formalization:** In tax-inclusive invoices, treating printed gross totals as the subtotal introduces double-taxation errors. Subtotal must be formalized as the **pre-tax taxable value**.

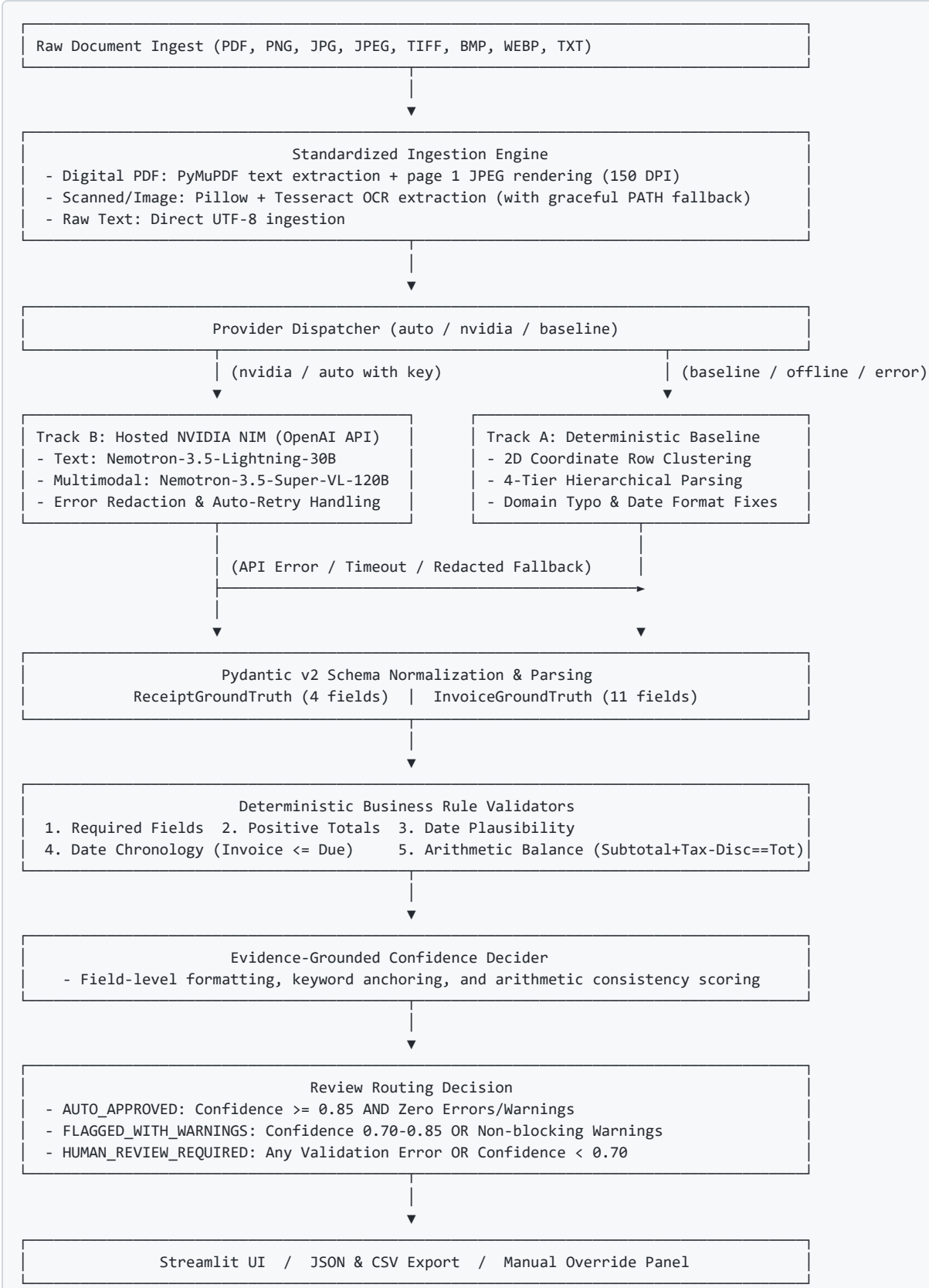
- **Explicit Zero-Tax Handling:** Real-world SaaS invoices frequently feature explicit 0% tax or $0.00 tax amounts (e.g., software exports). Systems treating zero as missing or falsy produce false validation errors.
- **Hosted API Vulnerability:** Hosted cloud APIs introduce network latency, rate limits, and outages. Mission-critical workflows require local offline fallbacks.

### 2.2 Core Design Goals & Engineering Trade-Offs

| Design Goal | Architectural Decision | Key Engineering Trade-Off |
|---|---|---|
| **Deterministic Verifiability** | Enforce Pydantic schemas and hard mathematical checks ( `Subtotal + Tax - Discount == Total` ). | Documents with unitemized fees fail validation and route to human review rather than guessing. |
| **Zero-Inference Principle** | Refuse to invent missing subtotals, tax components, or dates when documents lack them. | Lower straight-through processing on incomplete files; design objective is to minimize false auto-approvals by routing ambiguous or arithmetic-failing documents to human review. |
| **Dual-Engine Resilience** | Dispatch to NVIDIA NIM with automated fallback to local deterministic regex extractor. | Deterministic baseline lacks open-ended semantic comprehension on completely novel layouts. |
| **Credential Hygiene** | Safe resolution (Env -> Streamlit Secrets -> Baseline fallback) with regex error scrubbing. | Prevents credential leaks in diagnostic logs or UI exports; requires explicit configuration. |

## 3. Architecture and Document Ingestion

The DocExtract AI pipeline operates as a modular, feed-forward processing pipeline with clear boundaries between deterministic transformations and model inference.




### Ingestion Pipeline Summary

- **Digital PDFs:** Ingested via PyMuPDF ( `fitz` ). Evaluates a density check ( `< 15 * page_count` characters flags scanned PDFs). Renders page 1 to JPEG at 150 DPI for visual multimodal inspection. **Limitation:** Multimodal visual processing is currently restricted to page 1.
- **Scanned Images:** Ingested via Pillow and `pytesseract` . If Tesseract is not found on the host system PATH, the ingestor returns clean guidance without crashing ( `TESSERACT_GUIDANCE_MESSAGE` ). Compresses images to RGB JPEG buffer.

- **Supported Formats:** `.pdf`, `.png`, `.jpg`, `.jpeg`, `.tiff`, `.bmp`, `.webp`, `.txt`.

## 4. Extraction, Validation, and Review Routing

DocExtract AI unifies hosted foundation models, deterministic regex algorithms, schema enforcement, and review routing into a cohesive pipeline.

### 4.1 NVIDIA NIM Integration & Model Configurations

Hosted inference targets NVIDIA's OpenAI-compatible API (`https://integrate.api.nvidia.com/v1`) using `openai.OpenAI`:

| Model Identifier | Modality | Timeout | Max Tokens | Temp | Configured Role |
|---|---|---|---|---|---|
| `nvidia/nemotron-3.5-lightning-30b-a3b` | Text-Only | 45.0 s | 2048 | 0.0 | High-throughput text extractor from OCR text (Default). |
| `nvidia/nemotron-3.5-super-vl-120b-a12b` | Multimodal (Vision+Text) | 90.0 s | 4096 | 0.2 | Multimodal inspection of document image and OCR text. |
| `openai/gpt-oss-20b` / `120b` | Text-Only | 30.0 / 60.0 s | 1024 / 2048 | 0.0 | Hosted fallback text extractors. |

- **Multimodal vs. Text-Only Payload:** For text models, the user prompt transmits raw OCR text. For vision models with active image bytes, the payload includes both OCR text and a base64 image data URI (`...`).
- **Thinking Toggle & Retry:** Nemotron requests set `extra_body={"chat_template_kwargs": {"enable_thinking": False}}` to minimize latency. If the endpoint rejects `extra_body` with an `openai.BadRequestError`, the client strips it and automatically retries.
- **Exception Sanitization:** Catches auth, rate-limit, timeout, and schema errors, replaces API keys and bearer tokens with `[REDACTED_API_KEY]`, and returns `source="baseline_fallback"`.

### 4.2 Deterministic Regex Baseline & Provider Routing

- **Provider Routing Modes:**
  - `auto`: Prioritizes NVIDIA NIM if an API key is available; falls back to baseline on failure or missing key.
  - `nvidia`: Explicit NVIDIA NIM invocation with automatic fallback on failure.
  - `baseline` / `offline` / `regex`: 100% offline regex extractor (zero network calls).
  - Obsolete/unknown modes (e.g., `groq`, `gemini`): Safely routed directly to baseline.
- **Deterministic Baseline Mechanisms:**
  - *2D Coordinate Reconstruction:* Groups OCR tokens into rows when `Delta y <= 16 px` and sorts left-to-right (`x1`).
  - *Hierarchical Total Parsing:* Evaluates 4-tier keyword priority (`ROUNDED TOTAL` -> `TOTAL INCL. GST` -> `GRAND TOTAL` -> `TOTAL`) with adjacent lookahead, strictly excluding cash tenders, change, and tax summaries.
  - *Domain Typo Fixes:* Fixes OCR year errors (`701x -> 201x` and `261x -> 201x`) and collapses duplicate address commas.
  - *Pre-Tax Subtotal Mapping:* Scans for tabular GST headers (`gross amount`, `taxable value`, `sgst`, `cgst`, `igst`, `total`) and maps taxable value to `subtotal`.

### 4.3 Schemas and Business Validation Rules

The system enforces Pydantic v2 schemas: `ReceiptGroundTruth` (company, date, address, total) and `InvoiceGroundTruth` (vendor_name, invoice_date, due_date, subtotal, tax_amount, sgst_amount, cgst_amount, igst_amount, discount_amount, total_amount, currency).

Five deterministic validation rules are evaluated by `src/validators.py`:

1. **Required Fields:** Mandates vendor/company, date, and total (`ERROR` if missing).
2. **Positive Totals:** Grand total must be `> 0.0` (`ERROR`). Subtotal/tax/discount must be `>= 0.0`.
3. **Date Plausibility:** Parses 14 calendar formats; enforces year range `2000 <= year <= 2035` (`ERROR` if out of range).
4. **Date Chronology:** Enforces `invoice_date <= due_date` (`ERROR`). Skipped with `INFO` if either date is missing.
5. **Arithmetic Balance:** Verifies `Subtotal + Tax - Discount == Total` within `+/- 0.05` tolerance (`ERROR` on mismatch). If subtotal != total and tax is missing, flags for human review without inventing values.

### 4.4 Zero-Tax & Jurisdictional Tax Handling

- **Explicit Zero Tax ( `tax_amount == 0.0` ):** Preserved as numeric zero. When verified mathematically ( `Subtotal == Total` ), assigned high confidence (**0.98**). In standalone contexts without arithmetic, corroborated zero-tax ("0%", "tax exempt") receives **0.90** confidence; uncorroborated receives **0.70**. Negative tax ( `< 0.0` ) is penalized (**0.25**).
- **Absent Jurisdictional Taxes ( `sgst_amount` , `cgst_amount` , `igst_amount` ):** On non-INR invoices or documents without GST mentions, remaining null is valid, receives **1.0** confidence ("not applicable to non-GST document"), and displays as "N/A (Not Applicable)" in the UI rather than as an error.

### 4.5 Review Routing Outcomes

- `AUTO_APPROVED` : Composite confidence `>= 0.85` and zero validation errors/warnings.
- `FLAGGED_WITH_WARNINGS` : Composite confidence between `0.70` and `0.85` , or presence of non-blocking `WARNING` rules.
- `HUMAN_REVIEW_REQUIRED` : Any `ERROR` -level validation failure or composite confidence `< 0.70` .

> **IMPORTANT:**
>
> **Confidence vs. Correctness:** High confidence reflects consistent evidence signals (formatting, keyword anchors, arithmetic balance); it is **not** an authoritative guarantee that an extracted field matches ground truth.

---

## 5. Evaluation Methodology and Results

### 5.1 SROIE Dataset & Evaluation Protocol

The benchmark protocol evaluates the **ICDAR 2019 SROIE Dataset** (Scanned Receipts OCR and Information Extraction):

- **Dataset Size:** 437 clean, unique receipt image-label pairs containing all four fields ( `company` , `date` , `address` , `total` ).
- **Fixed Split Protocol (Seed 42):**
  - **Development Set (N = 349, 80%):** Used exclusively for error auditing, rule formulation, and benchmark comparison.
  - **Held-Out Test Set (N = 88, 20%):** Kept strictly unaccessed during development to prevent data snooping and tuning leakage.
- **Evaluation Metrics ( `src/evaluate.py` ):**
  - *Exact Match (%):* Strict character-for-character equality after trimming whitespace.
  - *Normalized Match (%):* Domain normalization (legal suffixes stripped, ISO dates, address punctuation collapsed, numeric floats rounded to 2 decimals; RapidFuzz `>= 85` for company, `>= 80` for address).
  - *RapidFuzz Similarity:* Token set ratio scored 0.0 to 100.0.
  - *Macro Averages:* Unweighted mean across all four target fields.

### 5.2 Development Set Benchmark Results (N = 349)

Evaluation comparing the baseline regex extractor against the improved deterministic pipeline:

| Target Field | Baseline Exact | Improved Exact | Baseline Norm | Improved Norm | Norm Diff (pp) | Baseline Fuzz | Improved Fuzz |
|---|---|---|---|---|---|---|---|
| **company** | 12.32% | **53.01%** | 39.26% | **82.81%** | **+43.55 pp** | 55.42 | **86.87** |
| **date** | 57.88% | **97.13%** | 68.19% | **97.42%** | **+29.23 pp** | 77.52 | **98.05** |
| **address** | 0.00% | **19.48%** | 69.05% | **91.69%** | **+22.64 pp** | 79.54 | **93.67** |
| **total** | 39.54% | **67.05%** | 52.44% | **84.53%** | **+32.09 pp** | 74.79 | **89.83** |
| **MACRO AVERAGE** | **27.44%** | **59.17%** | **57.23%** | **89.11%** | **+31.88 pp** | **71.82** | **92.11** |

- **Missing Prediction Recovery:** Missing date predictions dropped from 71 (20.34%) to 4 (1.15%), recovering **67 dates** through 2D coordinate grouping and OCR typo fixes.
- **Local Benchmark Latency & Cost:** Evaluates all 349 development records in **1.20 seconds total** (**3.43 ms/document**) with **$0.00 API cost**.

> **WARNING:**
>
> **Evaluation Scope & Held-Out Status:**
>
> 1. **Held-out evaluation is pending:** The split manifest `sroie_split_seed42.json` was generated in Google Colab and has not been synced locally. In accordance with evaluation integrity rules, **no held-out results are claimed**.
> 2. **Model Scope:** These figures evaluate the **deterministic extractor**, not NVIDIA NIM models across SROIE. Receipt performance does not guarantee identical invoice accuracy.
> 3. **Latency Distinction:** The 3.43 ms latency describes local CPU regex extraction, not NVIDIA NIM network API latency.

## 6. Testing, Security, and Reproducibility

### 6.1 Automated Test Suite Verification

The repository maintains an automated test suite executed via `pytest`:

```
============================ 96 passed in 3.45s ============================
```

- **Extraction & Ingestion (39 tests):** 29 tests in `test_nvidia_extractor.py` verifying API key resolution, secrets precedence, mocked Lightning/Super-VL payloads, retry on 400 Bad Request, mocked error fallbacks (auth, rate-limit, timeout, 404, JSON decode), and credential sanitization; 6 ingest tests (PDF/image/buffer); 4 baseline regex tests.
- **Business Validation & Schemas (25 tests):** 13 validator tests (arithmetic rules, chronology, zero tax, jurisdictional tax neutrality); 5 schema serialization tests; 7 Indian GST domain tests (pre-tax taxable subtotal, SGST/CGST separation).
- **Confidence & Synthetic Edge Cases (26 tests):** 9 confidence scoring and routing tests; 8 synthetic edge-case tests (injected arithmetic errors, inverted dates, missing totals); 6 SROIE improvement tests (2D clustering, priority parsing).
- **Evaluation & Smoke (6 tests):** 3 metric evaluation tests; 2 smoke tests; 1 benchmark CSV schema test.

> **NOTE:**
>
> **Mocked vs. Live Tests:** The 29 NVIDIA tests use mocked HTTP responses to verify request construction and error handling deterministically. They verify software correctness, not live endpoint uptime or model extraction accuracy.

### 6.2 Security & Credential Hygiene

- **Precedence Order:** (1) Explicit argument -> (2) `NVIDIA_API_KEY` environment variable -> (3) `st.secrets` (`.streamlit/secrets.toml` or Streamlit Cloud Secrets) -> (4) `None` (triggers offline baseline).
- **Secret Isolation:** `.streamlit/secrets.toml` is explicitly excluded from version control in `.gitignore`. A safe template `.streamlit/secrets.toml.example` is committed.
- **Credential Scrubbing:** `sanitize_error_message()` scrubs API keys (`nvapi-...`, `gsk-...`, `AIza...`) and bearer tokens from error strings.

### 6.3 Reproducible Execution Instructions

```
# Activate virtual environment and install dependencies
.venv\Scripts\Activate.ps1
pip install -r requirements.txt

# Run full test suite (96 tests)
pytest tests/ -v

# Launch interactive Streamlit interface
streamlit run app.py
```

## 7. Limitations and Future Work

### 7.1 Observed System Limitations

1. **OCR Degradation:** Ingestion relies on Tesseract when digital text layers are absent. Low-resolution mobile photos ( `< 150 DPI` ) or perspective skew directly degrade downstream extraction.
2. **Receipt Address Variability:** Varied visual layouts result in **19.48% exact match** on addresses, despite achieving **91.69% normalized match**.
3. **Company Registration Numbers in Ground Truth:** Ground truth often includes registration numbers (e.g., `POPULAR BOOK CO. (M) SDN BHD (139386 X)` ), capping company exact match at **53.01%** (while normalized match reaches **82.81%**).
4. **Hosted LLM Latency:** Multimodal NIM inference requires network round-trips and hosted model processing, with latency estimated at several seconds per document (typically 2 to 5 seconds depending on network load), compared to 3.43 ms for local deterministic extraction.
5. **Multi-Page Visual Processing:** PDF visual rendering is currently restricted to page 1.
6. **Host Dependency:** Scanned image OCR requires Tesseract installed on the host system PATH.

### 7.2 Prioritized Future Work

1. **Held-Out Benchmark Pass:** Sync `sroie_split_seed42.json` and execute a single, frozen held-out evaluation pass on the 88 held-out records.
2. **NVIDIA NIM Invoice Benchmark:** Establish a formal, reproducible benchmark evaluating Nemotron models on a standardized invoice dataset (e.g., DocVQA or Cord).
3. **2D Spatial Address Parsing:** Incorporate lightweight spatial graph parsing for complex multi-line addresses.
4. **Multi-Page Visual Inference:** Extend PDF image rendering across all document pages for multi-page invoices.
5. **FastAPI Microservice & Active Learning:** Decouple extraction and validation into an asynchronous REST microservice with an active learning review feedback loop.

### 7.3 Conversational Expense Intelligence Using RAG

> **NOTE:**
>
> **Implementation Status:** Proposed future work; this capability has not been implemented or evaluated in the current assessment prototype.

A high-value architectural evolution for DocExtract AI is extending the current document extraction and validation pipeline into a **Conversational Expense Intelligence Engine** using Retrieval-Augmented Generation (RAG). While the current prototype extracts and validates individual receipts and invoices in isolation, enterprise accounts payable and expense auditing require querying, aggregating, and verifying expenses across historical document collections.

#### Core Architecture & Deterministic Aggregation

- **Structured Metadata Persistence:** Validated records from `InvoiceGroundTruth` and `ReceiptGroundTruth` are persisted to an analytical datastore alongside key metadata: transaction date, merchant/vendor name, currency, line items, tax components, inferred expense category, and source-document URI references.
- **Strict Separation of Compute and Narrative:** To prevent generative hallucinations in financial summaries, an LLM must **never** be permitted to compute financial totals independently. Instead:
  1. *Deterministic Database Aggregation:* Date-aware filtering, categorical groupings, and mathematical calculations (e.g., sums, averages, min/max) are executed strictly via deterministic database queries and aggregation.
  2. *RAG Document Retrieval:* Dense and sparse vector retrieval indexes document text, OCR tokens, and contextual annotations to retrieve supporting vouchers, verify contextual invoice nuances, and synthesize natural-language explanations with grounded document citations.

#### Representative Query Scenarios

- "*How much did I spend on travelling last month?*" -> System parses date boundaries ( `2026-09-01` to `2026-09-30` ), executes deterministic query `SELECT SUM(total_amount) WHERE category = 'Travel'` , retrieves supporting airline/hotel receipts, and returns the exact total with linked citations.
- "*Compare my food expenses across the last three months.*" -> Executes grouped monthly aggregations deterministically, calculates month-over-month variance, and presents a comparative table supported by merchant-level citations.

- *"Find the receipt for my largest expense last month."* -> Executes `ORDER BY total_amount DESC LIMIT 1`, retrieves the specific receipt image and metadata, and presents the voucher with line-item verification.

### Required Enterprise Enablers

- **Expense Categorization:** Machine-learning tax and expense classification taxonomy (e.g., Travel, Meals, Software, Office Supplies) applied post-extraction.
- **Persistent Relational & Vector Storage:** PostgreSQL / pgvector hybrid datastore maintaining ACID guarantees for financial totals alongside embedding indices.
- **Date-Aware Filtering & Multi-Currency Normalization:** Calendar boundary normalization and conversion to baseline reporting currency via historical exchange rate tables.
- **Tenant-Level Access Controls:** Role-based access control (RBAC) ensuring users and property managers (e.g., *TenantEase*) only retrieve records within their authorized organization.

### Future Evaluation Criteria

When developed, this capability must be evaluated against five rigorous benchmarks:

1. *Numerical Accuracy (100% Target):* Zero hallucinated totals; all reported figures must match underlying ledger records exactly.
2. *Retrieval Relevance (Recall@K / MRR):* Precision and recall of retrieved vouchers supporting each user query.
3. *Citation Correctness:* Verifiable grounding of every claim against an authenticated document reference.
4. *Category Classification Accuracy (F1 Score):* Multi-class classification accuracy across expense taxonomies.
5. *Edge-Case Robustness:* Graceful handling of missing records, duplicate submissions, and ambiguous date ranges.

---

## 8. References

### Project Companion Reports & Core Files

- AI Opportunity Report (AI_OPPORTUNITY_REPORT.md) — Business alignment, 12 candidate opportunities, and prioritization matrix.
- Final Assessment Report (FINAL_ASSESSMENT_REPORT.md) — Baseline failure audit, engineering improvements, and SROIE benchmarks.
- Ingestion & Dispatcher: `src/ingest.py` | `src/extractor_dispatcher.py`
- Extraction Engines: `src/nvidia_extractor.py` | `src/baseline_regex.py`
- Validation & Routing: `src/validators.py` | `src/confidence.py` | `src/schemas.py`
- Evaluation & App: `src/evaluate.py` | `src/config.py` | `app.py`

### Benchmark Artifacts & External Documentation

- Development Summaries: `results/dev_baseline_summary.csv` | `results/dev_improved_summary.csv`
- ICDAR 2019 SROIE Challenge: Task 1 & Task 2 Standards
- NVIDIA NIM API Documentation: NVIDIA API Catalog
- Pydantic v2 Documentation: Pydantic Core